

Hermes vs OpenClaw

面向 Linux 内核 / 驱动 / BSP 团队的 AI Agent 技术评审

评审关注点：源码树作业、构建验证闭环、复杂缺陷分治、知识沉淀
适用对象：Kernel / Driver / DTS / Kconfig / Bring-up / Performance
结论预告：Hermes 更像进入研发现场的 engineering agent

```
$ rg "compatible" arch/ drivers/ dts/
```

```
$ make ARCH=arm64  
CROSS_COMPILE=aarch64-linux-gnu-
```

```
ld.lld: undefined reference to  
foo_clk_init
```

```
[ 3.821] probe defer: regulator not  
ready
```

```
$ git diff -- drivers/clock/foo.c
```

```
$ write RCA + patch plan + regression  
notes
```

核心评判标准：执行闭环深度

先定义评审问题：低层研发到底需要什么样的 AI Agent？

如果评审目标是 Linux 内核 / BSP 提效，评价标准应围绕真实工程动作，而不是消息入口数量

代码理解类

- 搜索结构体、宏、Kconfig、DTS 节点
- 跟踪 symbol 定义与调用路径
- 比较不同架构 / SoC 的实现差异

修改与验证类

- 小步 patch 驱动 / DTS / Kconfig / 文档
- 运行 make、脚本与检查工具
- 读取 warning / error / linker 报错

调试与定位类

- 分析 boot log、dmesg、oops、panic
- 追踪 probe defer / clock / regulator 依赖
- 定位性能回退与回归引入点

长周期协作

- 沉淀复现步骤与调试流程
- 保留平台约束与提交规范
- 输出 RCA 与回归记录

低层研发的典型闭环

查定义



改代码



跑构建



看日志



复盘沉淀

结论：评审重点应是“能否打穿查 - 改 - 编 - 测 - 记闭环”，而不是“能接入多少入口”。

从工作界面看：Hermes 更贴近源码树与工具链

内核/BSP 开发天然围绕仓库目录、shell、Makefile/Kconfig/DTS、日志与 patch 运转

Hermes：工作在仓库旁边

- 终端执行：make、clang/gcc、脚本、检查命令
- 文件操作：read_file / search_files / patch / write_file
- 任务编排：todo + delegate_task 适合复杂缺陷分治
- 知识沉淀：skill + memory 绑定项目约定与调试套路
- 输出物一体化：patch、RCA、回归记录、说明文档



更靠近执行环境

OpenClaw：平台触达能力强

- Gateway / 多渠道 inbox / voice / canvas
- 长期在线、入口丰富、助手平台化体验
- 多代理路由与多工作区隔离
- 更偏“助手在哪些界面可用”

对内核研发的含义

- 主战场仍是源码树与 shell
- 问题通常跨 DTS / driver / Kconfig / 日志
- 关键在于本地闭环，不在于移动端或语音
- 因此 Hermes 的工作形态更直接匹配需求

Hermes 对低层研发更有价值的能力栈

技术评审应关注：工具粒度、执行闭环、问题拆解能力，以及是否支持长期项目记忆

1 细粒度源码操作

按路径搜索、分页带行号读取、精准 patch，适合内核仓库的小步迭代。

2 构建与日志闭环

直接运行 make / script / checker，围绕 warning、link error、boot log 快速迭代。

3 复杂缺陷分治

并行查看 DTS、driver、Kconfig、diff 与日志，更适合跨层定位。

4 技能沉淀

可把 probe 排查顺序、交叉编译命令、提交流程固化为团队技能。

5 持久记忆

持续记住项目约定、平台限制与维护者偏好，适合长周期维护。

6 文档与实现一体化

代码、RCA、回归说明与 patch plan 可在同一上下文持续生成。

核心判断：Hermes 更像“驻留在代码仓里的工程代理”，而不是“部署在多入口上的个人助手”。

典型 Linux 内核 / BSP 场景：Hermes 更容易落地

下面这些问题都不是单纯问答场景，而是需要同时查看源码、配置、日志并迭代验证的工程问题

驱动 probe 失败

- 查 compatible、match table、clock/reset/regulator
- 结合 dmesg 与 probe 路径定位 -EPROBE_DEFER

设备树与驱动不匹配

- 核对 DTS / dtsi / binding / phandle 关系
- 把设备树、驱动和日志放进同一分析上下文

交叉编译 / 链接异常

- 围绕 undefined symbol、Kconfig 依赖快速追踪
- 小步 patch 后立即复验

性能回退分析

- 把 diff、热路径、锁竞争、配置变化拆成并行子任务
- 形成阶段化调查清单与复盘说明

上游提交流程辅助

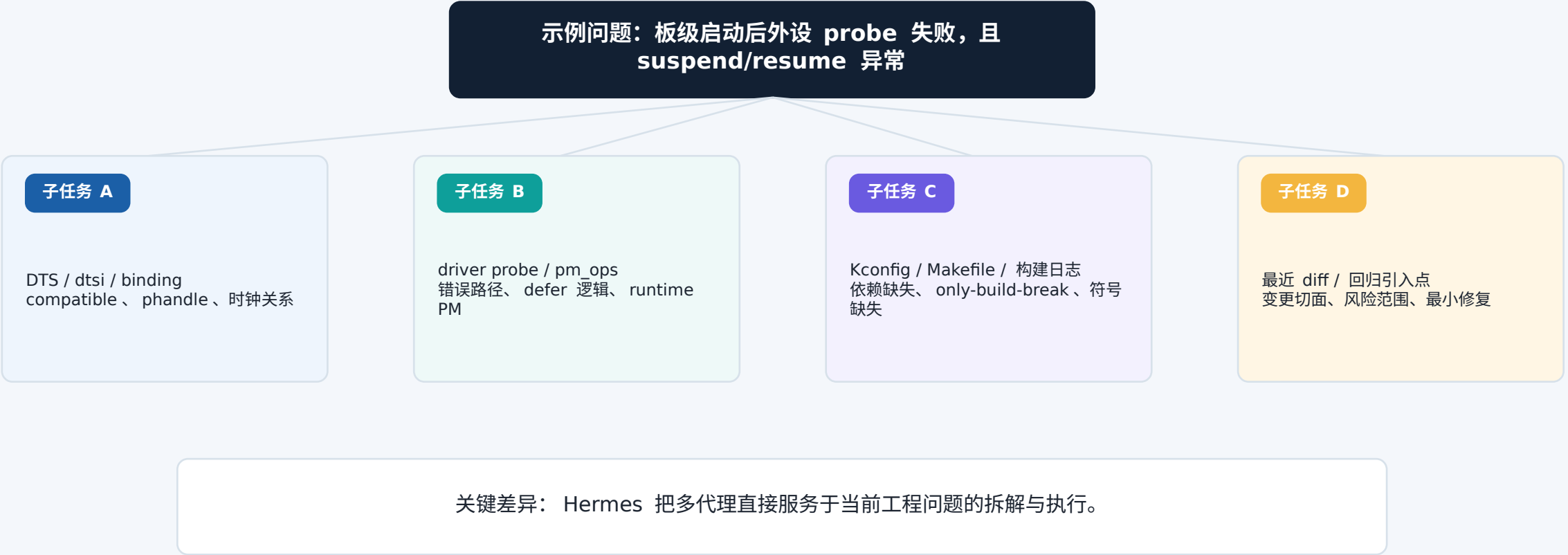
- 补 patch 切分、commit message、设计说明与回归记录
- 更适合沉淀 maintainer 偏好和提交流程

更多低层场景

- bring-up、suspend/resume、clock、regulator、runtime PM
- 共性是都依赖真实执行环境

复杂缺陷示例：Hermes 更适合做多维度分治

低层 bug 经常跨 DTS、driver、Kconfig、构建与 runtime 多个层次，一个视角往往看不清问题



表格化技术对比：哪个更利于内核 / BSP 团队？

这里的“优先”只针对 AI 辅助低层研发场景，不针对泛化个人助手市场

维度	Hermes	OpenClaw	本场景更优
设计中心	工程执行闭环	多渠道助手平台	Hermes
源码树作业	读 / 搜 / 改 / patch 一体化	不是主叙事中心	Hermes
终端 / 构建	shell / make / script / log	可支持，但非核心卖点	Hermes
复杂问题分治	更偏问题拆解	更偏代理路由	Hermes
经验沉淀	skill + memory	可做，但平台导向更强	Hermes
多入口触达	一般	极强	OpenClaw

结论：如果问题被严格限定为“AI 辅助 Linux 内核 / BSP 开发”，Hermes 的相对优势更清晰。

公平边界：OpenClaw 并不弱，只是优化目标不同

技术评审要避免“绝对优劣”的误判，应明确对比前提与适用边界

什么时候 OpenClaw 更值得选

- 目标是建设多渠道、长期在线的 AI 助手平台
- 需要 Telegram / Slack / Discord / WhatsApp 等统一接入
- 强调语音、移动端、Canvas 等交互形态
- 平台层的路由、在线化和触达能力是首要诉求

什么时候 Hermes 更值得选

- 目标是 Linux kernel / driver / BSP 团队提效
- 需要 AI 深入参与代码理解、patch、编译、日志分析
- 需要把调试套路、构建命令、维护者偏好沉淀下来
- 希望 AI 从“问答工具”升级为“工程执行工具”

换句话说：OpenClaw 更擅长“平台化触达”，Hermes 更擅长“工程化执行”。

技术评审结论

对 AI 辅助 Linux 内核 / BSP 开发，优先考虑工程执行型 Agent

Hermes 的关键胜点

- 更贴近 shell、源码树和构建系统
- 更适合细粒度 patch
- 更容易形成查 - 改 - 编 - 测 - 记闭环
- 更适合复杂低层问题分治
- 更适合沉淀平台知识与流程

OpenClaw 的强项

- 多渠道入口覆盖强
- 长期在线体验好
- 语音 / 移动端 / Canvas 丰富
- 更像成熟助手平台

推荐表述

- 平台化个人助手：优先参考 OpenClaw
- 内核 /BSP 团队提效：优先选择 Hermes 路线
- 决策应基于 workflow 匹配度

一句话总结：OpenClaw 更像“把 AI 放到很多地方去用”，Hermes 更像“把 AI 放进研发现场里干活”。